

Course - 2

Training set $\rightarrow$ dataset where machine is trained

Validation set $\rightarrow$ dataset where weights initialized

Testing set $\rightarrow$ final dataset

Bias and Variance

Train set 1% error 20% er

Val set 11% error 2% er

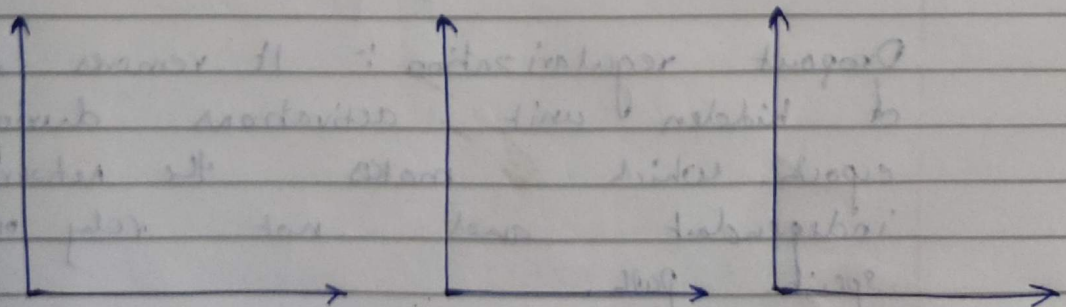
(high variance) (high bias)

High Bias $\rightarrow$ Bigger NN or Increase dataset

High variance $\rightarrow$ ~~Normal~~ Regularization

L2 Regularization $\rightarrow \|w\|_2^2 = \sum_{j=1}^{n \times m} w_j^2 = w^T w$

L1 Regularization $\rightarrow \frac{\lambda}{2m} \sum_{j=1}^{n \times m} |w_j| = \frac{\lambda}{2m} \|w\|_1$



Weight initialization

- * When weights are initialized as 0, all nodes of each layer receive same input and remain identical (Symmetry). Due to this activation functions remain 0 and network doesn't learn anything.
- * Large random weights are also very risky.
- * He initialization is standard if no initialization by PyTorch (ReLU).

Regularization:

It reduces variance by discouraging the NN by relying on large weights.

- * Large values of λ can make the model too simple (bias increase, smooth decision boundary).

Dropout regularization :- It removes a fraction of hidden unit activations during each epoch which makes the network independent and not rely on one specific path.

Keep prob = inverse of dropout

* Dropout mask should be used in forward as well as back prop. (same amount)

Other regularization methods:

- (i) More training data
- (ii) Data augmentation
- (iii) Model simplification

Input normalization

When input has very different scales, there can be issues in gradient descent, etc. Hence normalizing inputs is very important.

If each layer multiplies magnitude by c , after L layers the scale is c^L .

Gradient checking

Gradient checking compares back propagation gradients with numerical finite difference approximations.

$$\text{grad_approx} = [J(\theta + \epsilon) - J(\theta - \epsilon)] / (2\epsilon)$$

Centred difference is preferred over ~~approximate difference~~ one sided difference as error is smaller.

For multi-parameters $\Rightarrow$

diff. b/w grad & grad-approx is

$$g = \frac{\| \text{grad} - \text{grad_approx} \|_2}{(\| \text{grad} \|_2 + \| \text{grad_approx} \|_2)}$$

if difference $\approx 10^{-7}$ (valid)

if difference $\approx 10^{-3}$ (invalid)

* Batch & Mini batch gradient descent

Batch gradient descent $\Rightarrow$ all samples

advantage: Stable, Vectorized gradient

disadvantage: works slow on large dataset

Mini-batch gradient descent $\Rightarrow$ 1 - all samples

advantage: Fast vectorized gradient

disadvantage: no as such

RMS Prop

It keeps an exponential weighted average of squared gradients

Adam optimizer

It combines momentum 1st moment estimate with RMSprop's second-moment estimate.

* Adam is the best out of its alternatives

Learning rate decay

Large learning rate is useful early in training but it may cause repeated oscillation near a minimum. Step size is usually reduced so optimizer can make adjustments later.

Decay after every epoch

$$\alpha_{\text{epoch}} = \alpha_0 / (1 + \text{decay-rate} \times \text{epoch})$$

Hyper Parameter Tuning

Eg: α , β , no. of layers, ϵ (adam)

↓
Highest priority

↓
lowest priority

Random search > Grid search

Grids usually waste trials by testing various values of unimportant parameters. Random search assigns each parameter new values every run.

Coarse - to - Fine search

Don't spend all time in a useless region, broadly needs to be searched and then tested on.

Panda approach: Few models are watched and then params are tuned respectively it requires human monitoring

Caviar approach: Multiple models are trained in parallel and the best is selected.

Batch Normalization

It normalizes the pre-activation values of a layer across a mini-batch, then develops a new scale.

$$Z = \frac{x - \mu}{\sigma}$$

advantages:

- Training becomes faster & more stable
- Increases learning rate

~~System~~ classification

TensorFlow

tf.Tensor → Multidirectional data object
uses: inputs, activations & operations

tf.Constant → Immutable tensor value
uses: fixed data or constants

tf.Variable → Mutable state tracked by TensorFlow

Eg: Image pre-processing:

```
image = tf.cast (image, tf.float32) / 255  
image = - tf.reshape (image, [-1])
```

One hot encoding

$$[1 \ 2 \ 3 \ 0 \ 2 \ 1] = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Code example:

```
import numpy as np
import tensorflow as tf
```

```
coeffs = np.array([[1], [-20], [25]])
```

```
w = tf.Variable([0], dtype=tf.float32)
```

```
x = tf.placeholder(tf.float32, [3, 1])
```

```
cost = x[0][0] * w ** 2 + x[1][0] * w  
      + x[2][0]
```

```
train = tf.train.GradientDescentOptimizer(0.01)  
        .minimize(cost)
```

```
init = tf.global_variables_initializer()
```

```
session = tf.Session()
```

```
session.run(init)
```

```
print(session.run(w))
```

```
for i in range(1000):
```

```
    session.run(train, feed_dict)
```

```
print(session.run(w))
```